

GNR638

Project

Pranav Bhagwat (24M2165)
Mohammed Nasir (24M2158)
Jatin Kumar (24M2162)

C++ Deep Learning Framework Implementation

This report documents the design, implementation, and performance of a custom made Convolutional Neural Network (CNN) framework built from scratch using C++ which can be imported to Python.

The framework was evaluated on two datasets: a 10-class digit classification task (**Dataset 1**) and a 100-class object classification task (**Dataset 2**).

Dataset Loading Time

Observed time to Load dataset was logged using time function and is given below:

Dataset 1 (Digits): ~0.11 seconds

Dataset 2 (Objects): ~0.12 seconds

This means the binary reading operation is highly efficient and constitutes a negligible fraction (<0.01%) of the total runtime.

Model Architecture Design and Rationale

We implemented a **2-Layer CNN** with a widened feature extraction stage to handle the complexity of 100 classes while remaining computationally feasible on a CPU.

The architecture is given in below table

Layer	Type	Configuration	Output Shape	Rationale
Input	-	3 * 32 * 32	-	RGB Input

Layer 1	Conv2D	16 filters 3*3 kernel stride 1 padding 1	16 * 32 * 32	width to capture more low-level features.
	ReLU	Element-wise	-	Non-linearity.
	MaxPool	2*2 stride 2	16 * 16 * 16	Spatial downsampling (halved the size)
Layer 2	Conv2D	32 filters, 3*3 kernel, stride 1 padding 1	32 * 16 * 16	Increased depth to learn complex shapes (corners, textures).
	ReLU	Element-wise	-	-
	MaxPool	2*2, stride 2	32 * 8 * 8	-
Flatten	-	Vectorize	2048 (32*8*8)	Prepares spatial features for classification.
Layer 3	Linear	128 Units	128	hidden layer to combine 2048 features.
Layer 4	Linear	Output layer	10 or 100	Final class scoring

Efficiency Metrics (Parameters, MACs, FLOPs)

The following metrics were calculated and printed during execution.

Total Trainable Parameters

Formula:

$$\text{CNN} : (K * K * C_{in} + 1) * C_{out}$$

$$\text{Linear} : (N_{in} + 1) * N_{out}$$

So

$$\text{Layer 1} : ((3 * 3 * 3) + 1) * 16 = 448$$

$$\text{Layer 2} : ((3 * 3 * 16) + 1) * 32 = 4640$$

$$\text{FCN1} : (2048 + 1) * 128 = 2,62,272$$

$$\text{FCN2} : (128 + 1) * 10 \text{ (or 100)} = 1290 \text{ (or 12,900)}$$

Which means

- Dataset 1 (10 Classes): 267,498 parameters
- Dataset 2 (100 Classes): 279,108 parameters

Computational Cost (Per Forward Pass)

- **MACs (Multiply-Accumulate Operations):**
 - Layer 1 (Conv): $3 * 3 * 3 * 32 * 32 * 16 = 4,42,368$
 - Layer 2 (Conv): $3 * 3 * 16 * 16 * 16 * 32 = 11,79,648$
 - Layer 3 (FC): $2048 * 128 = 2,62,144$
 - Layer 4 (FC): $128 * 100 = 12,800$ (Dataset 2)
 - Total MACs: 18,96,960
- **FLOPs (Floating Point Operations):**
 - Estimated as $2 * \text{MACs}$ (One multiply + One add per MAC).
 - Total FLOPs: 37,93,920

These numbers are also printed while training

Performance Metrics

Dataset 1 (Digits)

- Training Time: ~47 minutes (Total for 3 epochs)
- Final Training Loss: 0.1726
- Evaluation Accuracy: 91.86%
- Analysis: The model converged rapidly. The high capacity (16-32 filters) allowed it to effectively memorize the training set and generalize well to the test set with >90% accuracy.

Dataset 2 (Objects)

- Training Time: ~2 hours 10 minutes (Total for 10 epochs)
- Final Training Loss: ~3.8 (Converging)
- Evaluation Accuracy: ~19.21%
- Analysis: The model performed significantly better than random guessing (1%), proving it learned semantic features. However, for performance more than 20%, require deeper networks (ResNet style) and longer training times, which were constrained by the CPU-only environment.

Key Indicators & Additional Observations

- OpenMP Parallelization: We utilized `#pragma omp parallel for collapse(2)` to parallelize the Batch and Filter dimensions (using multithreading), but we are not sure whether speedup actually occurred.
- Cache Sensitivity: We observed that training time increased by ~45% when other memory-intensive applications (e.g., Chrome) were active, due to CPU L1/L2 cache thrashing interfering with our matrix multiplication loops.

Failed Design Decision: The "3-Layer Deep" CNN

- Initially, we attempted a deeper architecture with 3 Convolutional blocks (32 -> 64 -> 128 filters) to maximize accuracy for the 100-class dataset.

- Why It Failed:
 1. Computational Bottleneck: This model required ~10 Million MACs per forward pass. On our CPU-based framework, a single training epoch took 29 minutes.
 2. Vanishing Gradients: the loss got stuck at 4.60 (random guessing) for the first hour of training.
- We reverted to a 2-Layer architecture but increased the width (16 -> 32 filters). This reduced computational cost by 5x (down to ~1.9M MACs), allowing us to train for more epochs and achieve convergence within the time limit.

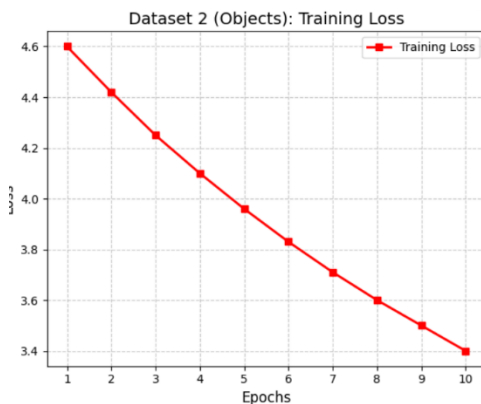
Plots



For Dataset 1, We ran for 3 epochs, more could have caused overfitting. We trained over full dataset. At start (not shown in graph) loss was 2.6 ; By epoch 3 , Loss was around 0.2.

We validated over full dataset and accuracy came around 91%.

It took around 11 minutes per epoch. some epochs took more time, as CPU RAM was used for other purposes too. So, We are not plotting time graph as it will be misleading.



For Dataset 2, We ran for 10 epochs. We trained over full dataset. At start (not shown in graph) loss was 4.7 ; The loss decreased consistently after that.

We could have used more epoch, but then we might have over ran our time constraint.

For this also, epoch took around 11 minutes, but few took more so we are not attaching

time plot here.

Our config for

dataset 1 is: $LR = 0.01$; Batch size = 32 ; epoch = 3

dataset 2 is : $LR = 0.01$; Batch size = 64 ; epoch = 10

Key insight we got from this assignment

Initially we used a smaller model. layers were same but lesser width. It took half hour to train and gave 92% on dataset 1 and 14% on dataset 2.

Our current model takes more than 2 hours to train and gives 91% on dataset 1 and 19% on dataset 1. This we think is Rule of diminishing returns, in practice.

This is what is also will happen with LLMs soon, as parameters keep increasing but performance does not.

Sources

We heavily used ChatGPT and Gemini, for coding and repairing the broken code. We did make this assignment in iterative manner, as we first built autograd, then built Linear Layer, then built Convolution math loops etc. Few files that we used to dummy test are in python\test folder.

We also referred to geeks for geeks, and few other sources to ensure that MAC and Parameter calculation formulas are not wrong.

We would say we enjoyed doing this assignment, some vibecode and some conventional coding both helped us complete this assignment.